In this chapter, you'll see:

- How doubles can isolate your code from your dependencies
- The differences between mocks, stubs, spies, and null objects
- How to add test double behavior to an existing Ruby object
- How to keep your doubles and your real objects in sync

CHAPTER 13

Understanding Test Doubles

In movies, a stunt double stands in for an actor, absorbing a punch or a fall when the actor can't or shouldn't do so. In test frameworks like RSpec, a *test double* fulfills the same role. It stands in for another object during testing.

You've used this concept before, in *Test Doubles: Mocks, Stubs, and Others*, on page 67. When you wrote your API unit specs, you treated the storage layer as if it were behaving exactly how you needed it to—even though that layer hadn't been written yet!

This ability to isolate parts of your system while you're testing them is superpowerful. With test doubles, you can:

- Exercise hard-to-reach code paths, such as error-handling code for a reliable third-party service
- Write specs for a layer of your system before you've built all its dependencies, as you did with the expense tracker
- Use an API while you're still designing it so that you can fix problems with the design before spending time on implementation
- Demonstrate how a component works *relative to its neighbors in the system*, which leads to less brittle tests

In this chapter, we're going to show you how to get started with rspec-mocks, RSpec's built-in library for creating test doubles. The techniques you learn here and in the next two chapters will make your specs faster and more resilient.

Types of Test Doubles

When we first introduced test doubles, we hinted that there are different names for doubles—mocks, spies, and so on. But we glossed over some of the differences. Let's take a closer look now.